

PROJECT REPORT OF CO-OP Project at Industry (Module-2)

ON

TaskMaster

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Sambhav Gupta

2210992236

Supervised By:

Mr. Ashish Chawla

Assistant Professor

Chitkara University



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

CONTENTS

S.No.	Title	Page No.
1	Acknowledgement	4
2	Introduction	5
3	Methodology	6
4	Tools and Technologies	7
5	Implementation	8 - 9
6	Major Findings	10 - 11
7	Conclusion and Future Scope	12
8	References	13

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “**TaskMaster**” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Mr. Ashish Chawla. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Sambhav Gupta

Date: 1 May, 2026

(2210992236.)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Mr. Ashish Chawla

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

I take this opportunity to express my profound gratitude and sincere thanks to my mentor, Mr. Ashish Chawla, for his invaluable guidance, constant supervision, and technical insights throughout the tenure of this project entitled "TASKMASTER: AI-Powered Student Task Reminder System." His encouragement and suggestions were instrumental in the successful completion of this work.

I am also deeply indebted to the Department of Computer Science and Engineering, Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, for providing the requisite infrastructure and resources that facilitated the realization of this project.

Furthermore, I wish to extend my heartfelt appreciation to the faculty members, colleagues, and friends who have directly or indirectly assisted and supported me during the development process.

Lastly, I am grateful to my family for their unwavering motivation and support throughout my academic endeavors.

Sambhav Gupta (2210992236)

1. INTRODUCTION

In today's academic environment, students are required to manage multiple responsibilities such as assignments, project submissions, examinations, internships, and extracurricular activities simultaneously. Managing these activities efficiently has become a major challenge for students due to increasing academic workload and strict deadlines. Traditional methods such as paper planners, sticky notes, or generic reminder applications often fail to provide an organized and intelligent approach for task management. Most existing reminder and task management systems only provide basic notification functionality and lack academic-specific features such as priority management, deadline tracking, and intelligent assistance. Students frequently miss deadlines because of poor planning, ineffective time management, and the inability to prioritize tasks properly. Furthermore, these systems do not provide actionable guidance or suggestions to help students complete their tasks effectively. To address these challenges, the project "TASKMASTER: AI-Powered Student Task Reminder System" has been developed. TaskMaster is a modern web-based application designed to help students organize, manage, and monitor their academic tasks efficiently. The system enables users to create tasks, assign deadlines, categorize priorities, and receive automated email reminders before deadlines. In addition, the project integrates Artificial Intelligence to generate task summaries and provide suggestions that assist students in planning and completing their work more effectively. The application is developed using modern technologies such as React, TypeScript, Supabase, PostgreSQL, and AWS SES for email notification services. The system follows a modular and scalable architecture to ensure security, reliability, and efficient performance. By combining automation and AI-powered assistance, TaskMaster aims to improve productivity, reduce missed deadlines, and enhance overall academic performance for students.

2. METHODOLOGY

The development of TASKMASTER followed a modular and systematic methodology, focusing on efficient implementation, scalability, and maintainability.

2.1 Requirement Analysis

Functional requirements included secure user authentication, task CRUD operations, deadline/priority management, automated email reminders, AI-generated summaries, and a tracking dashboard. Non-functional requirements focused on scalability, security, responsiveness, and fast performance.

2.2 System Design

A client-server architecture was adopted, separating the React/TypeScript frontend (UI, visualization) from the Supabase backend (authentication, database, notifications). The architecture consists of six core modules: UI, Authentication, Task Management, AI Assistance, Notification, and Database Management.

2.3 Frontend Development

The component-based UI was developed with React.js and TypeScript, utilizing Tailwind CSS and shadcn/ui for responsive design, covering registration, the task dashboard, forms, and progress tracking.

2.4 Backend and Database Integration

Supabase provides PostgreSQL database services and authentication. Task data is stored with a one-to-many user relationship, and Row-Level Security (RLS) restricts data access to each user's own tasks.

2.5 Automated Reminder System

A scheduling mechanism continuously monitors the database for upcoming deadlines. Automated email notifications are triggered and sent via AWS SES to prevent missed deadlines.

2.6 AI Assistance Module

The AI module is integrated to generate task summaries, actionable suggestions, and guidance, establishing the system as an intelligent productivity platform.

2.7 Testing and Validation

The system's reliability and functionality were verified using comprehensive testing methods, including Unit Testing, Integration Testing, User Interface Testing, and Database Validation Testing.

3. TOOLS AND TECHNOLOGIES

3.1 React.js

Used as the primary frontend library for building a dynamic and component-based user interface.

3.2 TypeScript

Ensures type safety and improves code maintainability throughout the development process.

3.3 Vite

A modern build tool used for fast development and optimized production bundling.

3.4 Tailwind CSS

A utility-first CSS framework employed for rapid UI development and responsive design.

3.5 shadcn/ui

Provides accessible and customizable UI components for a polished user experience.

3.6 Supabase

An open-source backend-as-a-service providing authentication, real-time database, and storage.

3.7 PostgreSQL

The relational database used for secure and structured task data management.

3.8 Supabase Auth

Used for implementing secure user login and session management.

3.9 TanStack Query

Used for efficient server state management, caching, and synchronization.

3.10 AWS SES

Amazon Simple Email Service is used for reliable delivery of automated task reminders.

3.11 pg_cron

A PostgreSQL extension used to schedule recurring reminder checks within the database.

3.12 AI Models

Language models integrated to provide intelligent task summaries and actionable suggestions.

4. IMPLEMENTATION

The implementation of the TASKMASTER: AI-Powered Student Task Reminder System was carried out using a modular development approach to ensure scalability, maintainability, security, and efficient performance. The system was developed as a modern web application consisting of frontend, backend, database, notification, and AI assistance modules.

4.1 Frontend Implementation

The frontend of the application was implemented using React.js and TypeScript. React provided a component-based architecture that enabled the creation of reusable and maintainable user interface components. TypeScript was used to ensure type safety and improve code quality. The frontend interface includes user registration and login pages, task dashboard, task creation and editing forms, priority and deadline management interface, and analytics and progress visualization. The application interface was styled using Tailwind CSS and shadcn/ui components to provide a responsive and modern user experience across different devices.

4.2 Backend Implementation

The backend services were implemented using Supabase, which provided authentication services, PostgreSQL database connectivity, real-time APIs, and secure cloud infrastructure. Supabase simplified backend development by handling authentication, database management, and secure API communication. User authentication was implemented using email and password login mechanisms.

4.3 Database Implementation

The application database was implemented using PostgreSQL. The database was designed to store user information and task-related details efficiently. The main database entities include Users Table, Tasks Table, Task Categories, and Task Priorities. A one-to-many relationship was established between users and tasks using unique UUIDs. Security mechanisms such as Row-Level Security (RLS) and secure authentication tokens ensured that users could only access their own tasks and information.

4.4 Task Management Module

The task management module was implemented to allow users to create, edit, and delete tasks, as well as update priorities and deadlines. Tasks were categorized into different priority levels such as Low, Medium, and High Priority to help students organize their academic workload efficiently.

4.5 Automated Reminder System

An automated reminder system was implemented to notify students about upcoming deadlines. The reminder system continuously monitors the task database and identifies tasks whose deadlines are approaching. The workflow includes fetching tasks, checking deadlines, and triggering scheduled email notifications via background processes.

4.6 AWS SES Integration

The system integrates Amazon Simple Email Service (AWS SES) for automated email delivery. AWS SES was configured to send reminder emails securely and reliably by connecting backend notification services and creating reminder email templates.

4.7 AI Assistance Module

The AI module was implemented to provide intelligent support for students. The AI functionality generates task summaries, actionable suggestions, and guidance for task completion by analyzing task descriptions and providing productivity recommendations.

4.8 State Management and Data Synchronization

TanStack Query was used for efficient server-state management and caching. This provided features like API data fetching, data caching, optimistic UI updates, and real-time synchronization to improve application performance.

4.9 Testing and Debugging

The application was tested thoroughly using unit, integration, UI, and database validation testing methods. Testing confirmed stable application performance across multiple devices and browsers.

4.10 Deployment and Performance Optimization

The application follows a cloud-based deployment approach using Supabase services. Performance optimization techniques included code splitting, optimized API requests, and lazy loading. The final implementation resulted in a secure, scalable, and intelligent student task management system.

5. MAJOR FINDINGS

5.1 Major Findings

During the development and testing phases of the project, several important findings were observed:

- Students face difficulty in managing multiple academic tasks simultaneously using traditional reminder systems.
- Generic task management applications lack academic-specific features such as assignment categorization, deadline prioritization, and intelligent assistance.
- Automated reminders significantly reduce the chances of missed deadlines, confirmed by implementation using `pg_cron` scheduling.
- AI-generated task suggestions improve task understanding and planning efficiency, moving the system beyond basic notification to intelligent assistance.
- Cloud-based deployment using Supabase improves scalability, reliability, and accessibility of the system, simplifying deployment and infrastructure management.

5.2 Outcomes of the Project

The project successfully delivered the following outcomes:

1. **Intelligent Task Management System:** A fully functional web-based application was developed allowing students to create, manage, prioritize, and efficiently organize academic activities. The system uses unique UUIDs for tasks and maintains a secure one-to-many relationship between users and tasks.
2. **Automated Reminder Functionality:** The system automatically sends reminder emails before deadlines, integrated securely using AWS SES. This functionality helps students stay informed and improves time management by actively preventing missed deadlines.
3. **AI-Powered Assistance:** The integrated AI module provides task summaries, actionable productivity suggestions, and planning recommendations, effectively transforming the system into an intelligent academic assistant.
4. **Secure User Management:** Secure user authentication is provided via Supabase Auth, and **Row-Level Security (RLS)** is enforced to guarantee user-specific data access and protection, ensuring high data integrity.
5. **Responsive User Interface:** The application provides a responsive and user-friendly interface built with React.js and Tailwind CSS, ensuring optimal user experience across desktop and mobile devices.

5.3 System Output

The developed system provides the following measurable outputs:

- User Authentication Dashboard and Task Management Interface for comprehensive task CRUD operations.
- Deadline and Priority Tracking with categorization into Low, Medium, and High priority levels.
- Automated Email Notifications delivered reliably via AWS SES, triggered by a pg_cron scheduling mechanism.
- Analytics Dashboard for visualizing completed and pending task progress.
- AI-Generated Recommendations providing intelligent guidance and planning support.

5.4 Performance Results

Testing confirmed the system's efficiency and stability:

- **Fast Page Loading:** Achieved through the use of the Vite build tool and optimized production bundling.
- **Smooth User Interaction:** Ensured by using **Optimistic UI updates** and effective caching provided by TanStack Query.
- **Secure Real-Time Data Synchronization:** Enabled by Supabase's real-time APIs and secured by JWT tokens.
- **Reliable Email Delivery** and **Stable Performance** confirmed across various testing scenarios and devices.

5.5 Benefits Achieved

The project implementation yielded significant academic benefits:

- Improved student productivity and enhanced time management skills.
- Better academic task organization and simplified task tracking.
- Significant **reduction in missed deadlines** due to proactive, automated reminders.
- Intelligent planning support through AI recommendations, which helps students prioritize and understand complex assignments.

6. CONCLUSION AND FUTURE SCOPE

Conclusion

The project “TASKMASTER: AI-Powered Student Task Reminder System” was successfully designed and implemented to address the critical challenges students face in managing academic tasks and deadlines. The system integrates secure user management, automated reminders via AWS SES, and AI-powered task assistance into a scalable and secure web application. By leveraging modern technologies like React.js, TypeScript, Supabase, and a modular architecture, the platform ensures high performance, maintainability, and data security using mechanisms like Row-Level Security and secure authentication tokens. TaskMaster successfully demonstrates the practical implementation of cloud-based services and AI to significantly improve student productivity and reduce instances of missed deadlines.

Future Scope

While the current system successfully achieves its primary objectives, several enhancements are planned to further improve functionality and user experience:

- 1. Mobile Application Development:** Develop dedicated native applications (using frameworks like React Native or Flutter) to provide an enhanced, on-the-go experience with local storage and platform-specific features.
- 2. Advanced AI Integration:** Integrate advanced **Large Language Models (LLMs)** to offer more granular, personalized productivity suggestions, intelligent scheduling based on task complexity, and context-aware guidance.
- 3. Voice Assistant Support:** Implement **voice-based task creation** and reminder functionality using speech recognition to allow hands-free interaction with the system.
- 4. Collaboration Features:** Introduce features for **group task management**, enabling students to track and coordinate team assignments and collaborative projects efficiently.
- 5. Calendar Integration:** Implement seamless, two-way synchronization with external calendars (e.g., Google Calendar, Microsoft Outlook) for centralized deadline management and scheduling.
- 6. Gamification and Engagement:** Integrate reward systems, badges, and achievement tracking to increase user motivation and long-term engagement with the platform.
- 7. Push Notifications:** Implement real-time browser and mobile push notifications for immediate alerts, serving as a faster and more proactive alternative to email reminders.

7. REFERENCES

1. React Official Documentation. Available at: <https://react.dev>
2. TypeScript Documentation. Available at: <https://www.typescriptlang.org/docs/>
3. Supabase Documentation. Available at: <https://supabase.com/docs>
4. PostgreSQL Documentation. Available at: <https://www.postgresql.org/docs/>
5. Amazon Web Services Documentation – AWS SES. Available at: <https://docs.aws.amazon.com/ses/>
6. Tailwind CSS Documentation. Available at: <https://tailwindcss.com/docs>
7. Vite Official Documentation. Available at: <https://vitejs.dev/guide/>
8. TanStack Query Documentation. Available at: <https://tanstack.com/query/latest>
9. shadcn/ui Documentation. Available at: <https://ui.shadcn.com>
10. Git Official Documentation. Available at: <https://git-scm.com/doc>
11. GitHub Documentation. Available at: <https://docs.github.com>